Email: John.dixon@hull.ac.uk
Meetings: https://tinyurl.com/jdixonhull
Feedback: https://tinyurl.com/jdhullfeedback
Office: RBB-315

Asynchronous Operations

Distributed Systems Programming

Dr John Dixon

UNIVERSITY of HULL

Overview

We are looking at these topics today:

- What Async is and how is it different from multithreading
- When we might want to perform asynchronous operations in the client
- When we might want to perform asynchronous operation on the server
- Async programming in C# (and JavaScript)
  - The async and await keywords and the Task type

What does asynchronous mean?

Not existing or occurring at the same time.

So, isn’t all non-multithreaded software asynchronous?

Confusingly, no. Synchronous code also does not run “at the same time” as other code (unless it is running on a different thread).

In synchronous code, tasks are executed one at a time in a **blocking manner**

- The program waits for the current task to finish before moving on to the next one.
- The code execution is sequential - each task must complete before the program can proceed to the next task

This is what we are used to when writing code.

What does asynchronous mean?

In a computer science context, asynchronous programming is:

A model where tasks are **non-blocking**

- An external task is initiated and allowed to run independently
- The program can continue to execute other tasks while the first task is still running
- When the external task finishes, the program will return to finish up any other work related to the task.

Notably, this does not mean that your software is doing more than one thing simultaneously – this is NOT multithreading.

- But it is allowing other external processes to do their work whilst continuing to execute other behaviours

UNIVERSITY of HULL

How is this different from Multi-Threading?

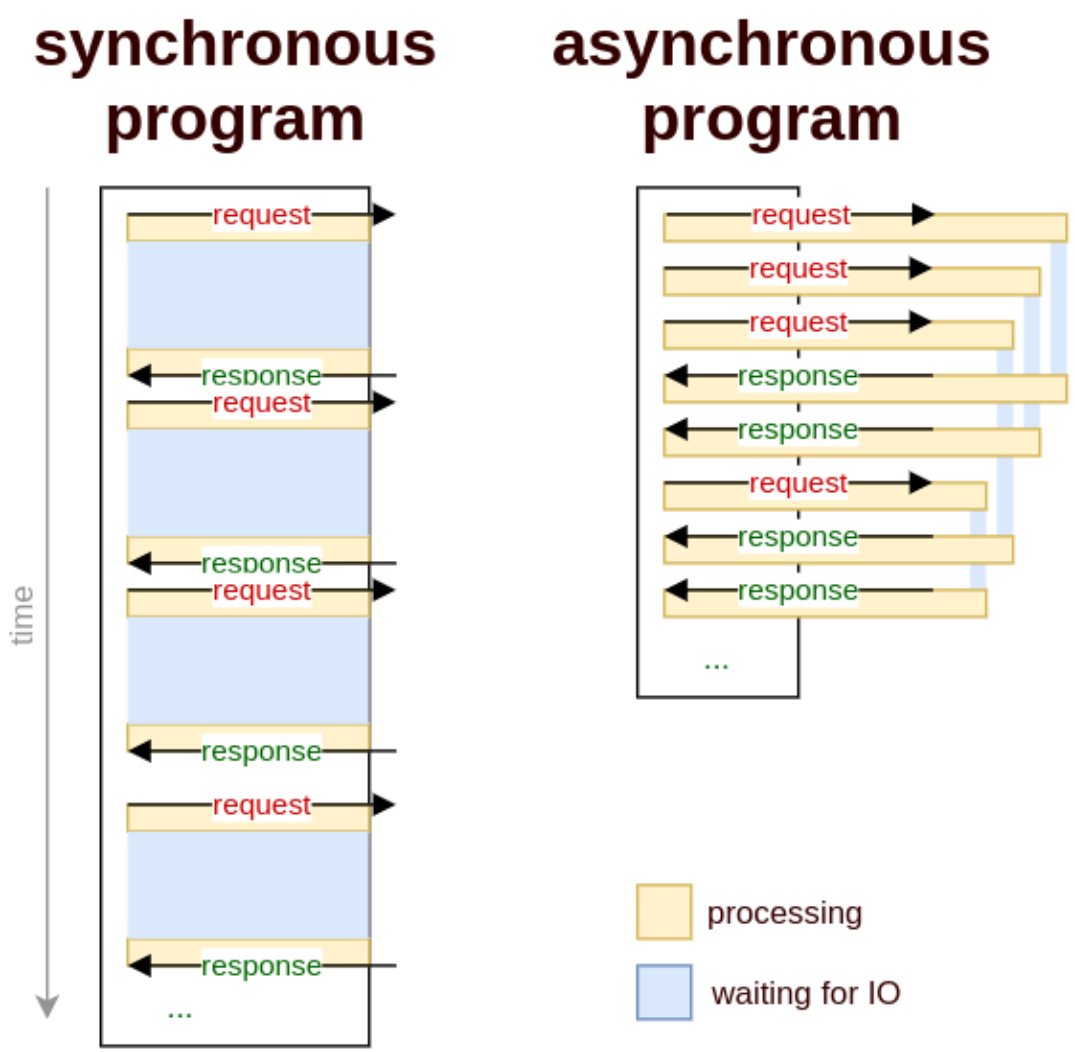
Multithreaded applications have code running ‘at the same time’ (concurrency).

We will consider the implications for this in a later lecture.

Asynchronous calls simply initiate an external operation and then continues to allow the execution of the code until the external operation completes.

When the operation completes, the runtime pauses the execution of the code and resumes from where the result of the external operation was being awaited. It completes any further work there and then, once that is done, un-pauses the execution of the paused code

Visually…


synchronous program
asynchronous program
time
request
response
request
response
request
response
request
response
...
request
request
request
response
response
request
response
response
...
processing
waiting for IO

Using Multi-Threading as part of Acync Programming!

Async programming REQUIRES code to be running outside the main thread.

(call is made to external code, your software continues whilst that code runs)

This could be I/O operations being conducted by the operating system

Or code running on a remote server

But it could also be **code running in a new worker/background thread that you have started**.

By combing async and multithreading you are giving more control over managing what happens after the completion of that work.

UNIVERSITY of HULL

Why do we want this?

We expect application responsiveness in our user interfaces

- Not getting greyed out and showing a spinner

We want to make efficient use of server resources

- If we can ask for two or more things we need, then collect together or display the responses when they are ready, we can offer a much more streamlined application

Asynchronous operations are particularly important in web and mobile applications, where network latency and unreliable connectivity can result in long wait times for users.

When do we want this?

Asynchronous operations are commonly used in situations where the operation could potentially take a long time to complete, such as network operations or disk I/O.

This means that almost all distributed applications are likely to benefit from some async programming when:

- Making a request or remote procedure call (waiting for a response)
- Loading data from a hard-drive
- Accessing a database
- Performing any function that is likely to take a long time (usually associated with loading data into memory or manipulating data)

Where do we want this? **Client**

Often we use async programming in the client

- Usually the client is requesting “stuff” from the server that might take some time
- We may want to be able to make lots of requests and remain responsive whilst we wait
  - If we are clever, we can use the time waiting for a response to interact with the user…

  E.g.

  1. Client initiates a request to get all records of data from a server
  2. Software asynchronously requests records from server
  3. Whilst waiting for the records to be received, ask the user for search parameters and for what they want to do to the data
  4. When the response has been received, load the records into local memory
  5. Once the user has finished adding their search parameters and identifying how they want to manipulate the data, the data is already ready to use. The user has not knowingly “waited”.

UNIVERSITY of HULL

Where do we want this? **Server**

Usually we multi-thread the server

- Each thread handles a single request / single socket

Sometimes we use async programming in the server

- Even though a thread may be in use for a request, we may need to perform multiple functions on the server before we can respond.

Examples include:

  - CRUD operations on the database
  - Making calls to other services
  - Processing middleware (pipeline) tasks that can be started whilst other pipeline tasks have not finished (e.g. logging whilst waiting for authorisation)

- We may be able to speed up our response time by asynchronously performing these functions

Why not start a new thread for each operation?

UNIVERSITY of HULL

How? **Await**

The await keyword is critical for asynchronous operations.

Calling method:

```
Code line 1: Some code
Code line 2: Call to async method
Code line 5: Some code
Code line 6: Some code

        -- external operation finishes --

Code line 9: Some code
```

Async method:

```
Code line 3: Some code
Code line 4: await the result of a call to an external operation
Code line 7: Some code
Code line 8: Some code
End of method
```

The await is allows the runtime to know when to return execution to the calling method and where to continue from in the async method when the external operation has completed.

UNIVERSITY of HULL

How? JavaScript

Because clients are often browser-based there is a LOT of literature on asynchronous programming in JavaScript.

```
async function fetchData() {

  var response = await fetch('https://mydomain.com/getsomedata');
  var data = await response.json();
  return data;
}

fetchData().then(data => {

    // Use the data. E.g. Manipulate the Document Object Model (DOM) to display the data to the user
});
```

The then keyword registers a callback to prevent the JavaScript from freezing until the response is received **and** converted from JSON

UNIVERSITY of HULL

How? C# (Distributed example using HttpClient)

```
static async Task GetData(HttpClient client)
{
    try
    {
        HttpResponseMessage response = await client.GetAsync("https://mydomain.com/getsomedata");

        if (response.IsSuccessStatusCode)
        {
            string json = await response.Content.ReadAsStringAsync();
            Console.WriteLine(json);
        }
        else
        {
            Console.WriteLine($"Request failed with status code {response.StatusCode}");
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Request failed with an exception: {ex.Message}");
    }
}
```

UNIVERSITY of HULL

Task Keyword

Notice that async C# methods return a Task

```
static async Task GetData(HttpClient client)
```

Task objects are useful for two purposes:

1. Task objects identify the status of the method, meaning that the calling code can identify what is happening inside the async method.
   - E.g. RanToCompletion, Faulted, Canceled

2. Task objects can be used to instigate an operation on a background thread in the threadpool. This operation can be awaited asynchronously in the foreground.

(See the MultiThreadExample in the workshops for Threadpool and Cancellation)

UNIVERSITY of HULL

Task Keyword

Async C# methods can also return a normal return type as well as returning a Task

```
static async Task<string> GetData(HttpClient client)
```

This async method will return a string.

Until the Task has finished running, there is no string returned – but the task is still able to be inspected by the calling method to find out what is going on inside.

UNIVERSITY of HULL

Calling Async methods from Synchronous Code

Often, we will still need to wait for some asynchronous operation to finish before we can continue beyond a certain point.

E.g. We asynchronously call 5 long-running operations, but we can not do anything with them until they all return.

We can use the Task.Wait() methods to arrange this synchronisation

```
GetData(HttpClient client).Wait();
```

Remember that GetData returned a Task. We are calling Wait() on that Task.

This pauses the execution in the calling thread until GetData is finished.

Calling Async methods from Synchronous Code

HEY! WE JUST MADE THAT ASYNC CALL SYNCHRONOUS! ☹

Yes, we did. Sometimes you need to do that.
More often, we want to do something like this:

```
Task taskOne = GetData(HttpClient client);
Task taskTwo = GetData(HttpClient client);
Task taskThree = GetData(HttpClient client);

taskOne.Wait();
taskTwo.Wait();
taskThree.Wait();
```

UNIVERSITY of HULL

Calling Async methods from Synchronous Code

We can achieve the same thing by using the static WaitAll behaviour inside Task:

```
Task[] tasks = new Task[3];
tasks[0] = GetData(HttpClient client);
tasks[1] = GetData(HttpClient client);
tasks[2] = GetData(HttpClient client);

Task.WaitAll(tasks);
```

There are other interesting behaviours like: `WaitAny`, `WaitAsync`, `WhenAll`, `WhenAny` and `Yeild` to learn about too!

https://learn.microsoft.com/en-us/dotnet/api/system.threading.tasks.task

UNIVERSITY of HULL

Further Reading

Microsoft:

https://learn.microsoft.com/en-us/dotnet/csharp/asynchronous-programming/

https://learn.microsoft.com/en-us/dotnet/api/system.threading.tasks

https://learn.microsoft.com/en-us/dotnet/api/system.net.http

Books:

Griffiths, Ian. Programming C# 8.0: Build Cloud, Web, and Desktop Applications. Japan, O'Reilly

Kelhini, Faraz K.. Modern Asynchronous JavaScript: Tackle Complex Async Tasks with Less Code, The Pragmatic Programmers, Pragmatic Bookshelf

UNIVERSITY of HULL

Reflection

In this session we've looked at:

- What Async is and how is it different from multithreading
- When we might want to perform asynchronous operations in the client
- When we might want to perform asynchronous operation on the server
- Async programming in C# (and JavaScript)

In the next session we will look at Remote Procedure Calling

UNIVERSITY of HULL